



SAPIENZA
UNIVERSITÀ DI ROMA

Studio comparativo di modelli NLP per la classificazione della rilevanza di articoli scientifici

Facoltà di Ingegneria dell'informazione, informatica e statistica
Corso di Laurea in Informatica

Candidato

Alessandro Gautieri
Matricola 2041850

A handwritten signature in black ink, reading "Alessandro Gautieri".

Relatore

Prof. Alessandro Checco

A handwritten signature in black ink, reading "Alessandro Checco".

Anno Accademico 2024/2025

**Studio comparativo di modelli NLP per la classificazione della rilevanza di
articoli scientifici**

Tesi di Laurea. Sapienza – Università di Roma

© 2025 Alessandro Gautieri. Tutti i diritti riservati

Questa tesi è stata composta con L^AT_EX e la classe Sapthesis.

Email dell'autore: gautieri.2041850@studenti.uniroma1.it

*«Chiedersi se un computer possa pensare è tanto interessante
quanto chiedersi se un sottomarino possa nuotare.»*

— Edsger W. Dijkstra

Sommario

La peer review è un processo essenziale per garantire la qualità e l'affidabilità delle pubblicazioni scientifiche. Tuttavia, è spesso affetta da inefficienze, incoerenze e costi elevati, dovuti al tempo e alle risorse richieste per valutare manualmente la rilevanza e il merito dei lavori proposti. Questa tesi affronta la sfida di automatizzare parte di questo processo attraverso l'utilizzo di modelli avanzati di Natural Language Processing (NLP), con l'obiettivo di sviluppare un modello in grado di predire automaticamente la pertinenza di un articolo rispetto a una conferenza scientifica specifica.

Il sistema proposto analizza il titolo, l'abstract e gli argomenti della conferenza target per determinare se il paper sia coerente con il contesto editoriale previsto, suggerendo eventualmente conferenze alternative più appropriate. Per la costruzione del dataset di addestramento, sono stati integrati e riprocessati dati provenienti da PeerRead, PeerSum e arXiv, e le etichette di rilevanza sono state generate automaticamente mediante Large Language Models (LLM), in particolare LLaMA 3. È stata posta grande attenzione a bilanciare la dimensione del dataset con la taglia del modello, evitando architetture sovradimensionate che avrebbero portato a overfitting e a un uso inefficiente delle risorse computazionali.

Diversi modelli NLP, tra cui BERT, DeBERTa e Mistral, sono stati addestrati e confrontati, mostrando risultati promettenti in termini di accuratezza e robustezza. È stata inoltre sviluppata una pipeline completa per la classificazione e il suggerimento di conferenze.

I risultati sperimentali confermano l'efficacia dell'approccio supervisionato e il potenziale dell'intelligenza artificiale nel supportare il processo di peer review, aprendo nuove prospettive per l'integrazione di sistemi NLP nel workflow editoriale delle conferenze accademiche.

Indice

1	Introduzione	1
1.1	Contesto e motivazioni	1
1.2	Cos'è la peer-review e perché serve	2
1.3	Obiettivi e contributi	2
1.4	Stato dell'arte	2
1.5	Fine-tuning dei modelli di linguaggio	3
1.6	Struttura della tesi	4
2	Dataset e Preprocessing	5
2.1	Descrizione generale e obiettivi	5
2.2	Classificazione automatica della rilevanza tramite LLaMA 3	5
2.2.1	Dataset originali e struttura iniziale	5
2.2.2	Strategia di classificazione e prompt specifico per LLaMA 3	6
2.2.3	Problemi incontrati durante l'annotazione e soluzioni adottate	7
2.2.4	Risultato della classificazione automatica	7
2.2.5	Validazione del dataset supervisionato finale	7
2.3	Costruzione del dataset supervisionato finale	9
2.3.1	Unione, normalizzazione e pulizia dei dati	9
2.3.2	Arricchimento dei dati tramite argomenti delle conferenze	9
2.3.3	Strutturazione testo di input e tokenizzazione	9
2.3.4	Suddivisione in set di training e test	9
2.4	Conclusione del processo	10
3	Modellazione e Training	11
3.1	Iperparametri principali	11
3.2	BiLSTM (Bidirectional Long Short-Term Memory)	12
3.2.1	Reti neurali ricorrenti e principio LSTM	12
3.2.2	Struttura BiLSTM e vantaggi	13
3.2.3	Implementazione e iperparametri nel progetto	13
3.3	BERT Large	14
3.3.1	Dalle Reti Neurali Ricorrenti ai Transformer	14
3.3.2	Il meccanismo di Self-Attention	15
3.3.3	Architettura di BERT	15
3.3.4	Fine-tuning nel nostro progetto	16
3.4	DeBERTa: Disentangled Attention	17
3.4.1	Motivazione e differenze rispetto a BERT	17
3.4.2	Formula semplificata della disentangled attention	17
3.4.3	Relative Position Bias	17
3.4.4	Implementazione e iperparametri nel nostro progetto	18
3.4.5	Confronto tra BERT e DeBERTa	18

3.5	Mistral: Multi-Query Attention e sequenze lunghe	19
3.5.1	Struttura generale del modello	19
3.5.2	Vantaggi pratici nella ricerca	19
3.5.3	Iperparametri utilizzati nel training	19
3.5.4	Confronto con gli altri modelli	20
3.6	Conclusioni del capitolo	21
4	Esperimenti e risultati	22
4.1	Valutazione dei modelli e risultati del fine-tuning	22
4.1.1	Setup sperimentale per il fine-tuning	22
4.1.2	Metriche di valutazione	23
4.1.3	Risultati comparativi	23
4.2	Perché Mistral-7b performa peggio di DeBERTa?	24
4.2.1	Spiegazione delle Chinchilla Scaling Laws	24
4.2.2	I nostri numeri	24
5	Pipeline finale di suggerimento conferenze basata su classificazione e retrieval	26
5.1	Introduzione	26
5.2	Pipeline Originale: Design e Limiti	26
5.2.1	Problemi Riscontrati	27
5.3	Architettura Two-Step	27
5.3.1	Fase 1 – Classificazione di Rilevanza	28
5.3.2	Fase 2 – Estrazione Topic e Retrieval	28
5.4	Pipeline Completa: Algoritmo	29
5.5	Risultati Sperimentali	30
5.6	Discussione	30
6	Conclusioni e Lavori Futuri	31
6.1	Conclusioni	31
6.2	Considerazioni sul Data Leakage	31
6.2.1	Cos'è il Data Leakage	31
6.2.2	Impatto nel nostro esperimento	31
6.3	Affidabilità del Suggerimento di Conferenze	32
6.4	Altre Problematiche e Limitazioni	32
6.4.1	Dipendenza dalla qualità dell'annotazione automatica	32
6.4.2	Limiti di generalizzazione	33
6.4.3	Bias introdotti dai LLM	33
6.5	Valutazione Zero-Shot con Gemini Flash	33
6.5.1	Il modello utilizzato: Gemini 1.5 Flash	33
6.5.2	Prompt utilizzato per Gemini	33
6.5.3	Risultati ottenuti con Gemini	34
6.5.4	Confronto con modelli dopo il fine-tuning	34
6.5.5	Discussione	34
6.6	Lavori futuri	35
	Bibliografia	36
	Ringraziamenti	40

Capitolo 1

Introduzione

1.1 Contesto e motivazioni

La *peer-review* è la spina dorsale del sistema scientifico: un passaggio obbligato in cui le idee e le ricerche di un autore sono sottoposte al giudizio di colleghi esperti prima di diventare conoscenza “certificata”. Negli ultimi anni il numero di sottomissioni alle principali conferenze di *computer science* è cresciuto in modo esponenziale – NeurIPS ha superato le 19 000 submission nel 2024, mantenendo un tasso di accettazione intorno al 20 % [1]. Ogni articolo richiede almeno tre reviewer volontari, chiamati a leggere decine di manoscritti in poche settimane. Il risultato? Affaticamento, giudizi talvolta incoerenti e rigetto di lavori che avrebbero potuto trovare conferenze più adatte.

Parallelamente, i **Large Language Models** (LLM) – da GPT-4 a LLaMA – hanno mostrato di saper riassumere, classificare e persino *ragionare* sul testo con prestazioni sempre più vicine a quelle umane. Questo scenario apre una domanda naturale: *possiamo usare i LLM per rendere più snello e mirato il primo filtro della peer-review?*

In questa tesi proviamo a farlo, con due obiettivi concreti:

- (i) **Alleggerire il carico dei reviewer**: un modello addestrato tramite fine-tuning pre-classifica i paper come **Rilevanti** o **Non rilevanti** per la conferenza a cui sono stati inviati.
- (ii) **Suggerire alternative intelligenti**: se un articolo risulta fuori tema per una data conferenza, il sistema propone automaticamente una conferenza diversa meglio allineata ai suoi argomenti, trasformandosi in un piccolo *assistente editoriale*.

L’idea non è sostituire il giudizio umano, ma fornire uno strumento che possa far risparmiare tempo a editor e reviewer, migliorando al contempo l’esperienza degli autori, che ricevono un feedback immediato sulla pertinenza della loro ricerca.

1.2 Cos'è la peer-review e perché serve

La *revisione paritaria* (peer review) è un processo in cui esperti del settore valutano la qualità scientifica di un manoscritto prima della pubblicazione. Essa assicura:

1. validazione dei metodi e dei risultati,
2. coerenza con lo stato dell'arte,
3. assenza di errori grossolani o plagio,
4. selezione dei lavori più pertinenti alla conferenza o alla rivista.

Con l'aumento esponenziale delle sottomissioni, il carico sui reviewer è diventato critico, automatizzare le fasi preliminari può ridurre tempi ed errori.

1.3 Obiettivi e contributi

Gli obiettivi principali del lavoro sono quattro:

1. costruire un dataset annotato in modo semiautomatico con l'ausilio di LLM,
2. addestrare (e confrontare) modelli Transformer addestrati sul dominio,
3. progettare una pipeline *two-step* con modulo di suggerimento di conferenze,
4. valutare le prestazioni con metriche standard e analizzare criticità (data leakage, rumore nelle etichette, bias, ecc.).

I contributi possono essere riassunti in:

- **Dataset** semi-curato da circa 10000 abstract di articoli con etichette bilanciate,
- **Fine-tuning** mirato di BiLSTM, BERT-Large, DeBERTa e Mistral-7B,
- **Sistema di raccomandazione** conferenze basato su similarità semantica,
- **Benchmark** comparativo con esperimento *zero-shot* (Gemini 1.5 Flash).

1.4 Stato dell'arte

Negli ultimi anni, l'impiego dell'**intelligenza artificiale** nel processo di **peer-review** scientifica ha attratto crescente interesse, soprattutto con l'avvento dei **Large Language Models (LLM)**. Questi modelli, come GPT-4 o LLaMA, hanno dimostrato una notevole capacità di generare testo coerente e valutazioni testuali complesse, spingendo numerosi ricercatori a valutarne l'impiego nei workflow editoriali.

Una prima applicazione dell'IA riguarda lo **screening automatico dei manoscritti**, con l'obiettivo di filtrare paper esplicitamente fuori tema o privi dei requisiti minimi formali e metodologici [2, 3]. Alcune conferenze hanno già adottato strumenti basati su modelli NLP per il controllo preliminare di qualità, plagio o attinenza al tema.

Più recentemente, l'attenzione si è spostata sulla possibilità che **gli stessi revisori utilizzino LLM per redigere o migliorare le proprie review**. Uno studio su migliaia di recensioni ha evidenziato che tra il 7% e il 17% dei testi presentavano segnali compatibili con la generazione automatica [4]. Sebbene ciò possa alleggerire il carico, emergono forti preoccupazioni su **trasparenza, responsabilità e bias** introdotti [5, 6].

Parallelamente, sono state proposte strategie in cui i LLM agiscono come **strumenti assistivi** per editor e reviewer. Approcci *human-in-the-loop* consentono al revisore umano di ricevere suggerimenti generati da LLM, che possono essere valutati e modificati [7]. Questi sistemi possono accelerare la revisione dei paper più semplici, ma faticano nel cogliere aspetti scientifici profondi come **rigore metodologico e originalità**.

Infine, alcuni studi mostrano che i LLM tendono a essere **eccessivamente indulgenti**, aumentando il tasso di accettazione dei paper valutati con il loro supporto, con effetti potenzialmente falsificati sull'equità del processo [8].

Nel complesso, la letteratura suggerisce che l'IA ha il potenziale per **supportare**, ma non **sostituire**, il giudizio umano nella peer-review. I sistemi più efficaci sono quelli che affiancano l'uomo, fornendo **riassunti, suggerimenti stilistici o alert automatici**, ma lasciando agli esperti la valutazione scientifica vera e propria.

1.5 Fine-tuning dei modelli di linguaggio

Un *Large Language Model* (LLM) pre-addestrato può essere visto come un'enciclopedia tascabile: conosce «un po' di tutto», ma non è specializzato sul nostro problema. Il **fine-tuning** serve proprio a questo: trasformare la conoscenza generalista del modello in competenza mirata sulla rilevanza dei paper.

Come funziona, in parole semplici

1. **Si prepara un piccolo set di esempio.** Nel nostro caso 9 750 record contenenti *titolo, abstract, nome conferenza* (con i relativi argomenti) e l'etichetta binaria *Rilevante/Non rilevante*.
2. **Si “mostrano” questi esempi al modello.** Durante la retro-propagazione il modello regola i pesi interni per abbassare l'errore di classificazione.

Perché è meglio del solo prompting

Con il prompting «zero-shot», ovvero senza fine-tuning specifico, il modello tenta di rispondere basandosi su conoscenze generiche, spesso scivolando in risposte incoerenti o contraddittorie. Dopo il fine-tuning, invece, il modello:

- comprende il lessico tipico dei paper scientifici (es. “baseline”, “ablation study”),
- diventa più stabile—basta un singolo prompt “asciutto” per ottenere una predizione affidabile,
- riduce drasticamente i falsi positivi perché ha visto esempi concreti di cosa *non* è pertinente a una conferenza.

Tre strategie adottate nella nostra ricerca

- a) **Fine-tuning completo** Tutti i pesi del modello vengono aggiornati (scelto per BERT-Large).
- b) **Adapter layer** Si aggiungono piccoli blocchi addestrabili dentro il modello, lasciando congelati gli altri pesi (impiegato per DeBERTa v3).
- c) **LoRA** [9] Si iniettano matrici a basso rango—pochi parametri—su alcune proiezioni chiave (usato su Mistral-7B quantizzato).

Tutte le impostazioni pratiche—learning rate, scheduler, dimensione del batch—sono descritte nel Cap. 3.

1.6 Struttura della tesi

Il resto del lavoro è organizzato come segue:

Il **Cap. 2** descrive la costruzione e la pulizia del dataset.

Il **Cap. 3** illustra l'architettura dei modelli e i dettagli di fine-tuning.

Il **Cap. 4** presenta i risultati sperimentali e il confronto tra i vari modelli.

Il **Cap. 5** introduce il modulo di suggerimento conferenze.

Il **Cap. 6** riassume conclusioni, limiti, problematiche e possibili lavori futuri.

Capitolo 2

Dataset e Preprocessing

2.1 Descrizione generale e obiettivi

L'obiettivo principale di questa tesi è sviluppare un sistema basato su modelli di linguaggio allenati tramite fine-tuning, in grado di assistere il processo di peer review scientifica attraverso l'analisi automatica dei contenuti dei paper. Per addestrare e validare tali modelli è stata necessaria la costruzione di un dataset dedicato, integrando fonti preesistenti e applicando tecniche avanzate di etichettatura semiautomatica per stabilire la rilevanza dei paper rispetto alle conferenze a cui erano stati originariamente inviati.

La costruzione del dataset ha richiesto un accurato processo di preprocessing, articolato in due fasi principali:

- **Classificazione automatica della rilevanza tramite LLaMA 3:** Creazione di un dataset iniziale etichettato automaticamente analizzando le review originali dei paper.
- **Costruzione del dataset supervisionato finale:** Utilizzo del dataset annotato per addestrare modelli capaci di prevedere la rilevanza di un paper a partire dal suo titolo, abstract e dagli argomenti della conferenza di destinazione.

2.2 Classificazione automatica della rilevanza tramite LLaMA 3

Il primo passo del progetto è stato quindi generare un dataset annotato in maniera semiautomatica, necessario per l'addestramento successivo dei modelli supervisionati. Poiché nessun dataset originale conteneva esplicitamente etichette di rilevanza rispetto agli argomenti delle conferenze, è stato introdotto un approccio innovativo basato su Large Language Models (LLM), in particolare il modello LLaMA 3.1 8B Instruct [10], selezionato per il bilanciamento tra capacità di comprensione semantica e performance computazionali.

2.2.1 Dataset originali e struttura iniziale

I dataset inizialmente disponibili erano eterogenei e strutturati nel seguente modo:

- **PeerRead** [11]: include paper con review complete e informazioni binarie di accettazione o rifiuto.

- **PeerSum** [12]: comprende una vasta raccolta di paper associati a review e decisioni editoriali.
- **arXiv**: costituito da articoli scientifici suddivisi per aree tematiche, privi di review ma dotati di metadati essenziali.

Ogni record del dataset è stato normalizzato in formato JSON, contenendo:

- identificativo unico del paper;
- titolo del paper;
- abstract del paper;
- testo completo della review (se presente);
- decisione editoriale (accettato, rifiutato, non disponibile);
- nome completo della conferenza con anno di riferimento.

2.2.2 Strategia di classificazione e prompt specifico per LLaMA 3

Per garantire un'annotazione coerente e accurata della rilevanza, è stato sviluppato un prompt dettagliato, ottimizzato tramite numerose iterazioni e test preliminari. Tale prompt guidava esplicitamente il modello LLaMA nell'attribuzione delle etichette di rilevanza, chiarendo i criteri decisionali:

- **Rilevante**: Il paper è coerente con gli argomenti della conferenza, indipendentemente dalla decisione editoriale finale.
- **Non Rilevante**: Il paper è stato esplicitamente rifiutato per mancanza di pertinenza agli argomenti della conferenza.
- **Non determinato**: Le informazioni disponibili non permettono di stabilire chiaramente la rilevanza del paper.

Prompt per la classificazione della rilevanza

Il prompt definitivo utilizzato per il modello LLaMA 3 è stato il seguente:

*“Hai a disposizione una review e l'informazione sulla decisione editoriale di un paper scientifico inviato a una conferenza. Se il paper è stato accettato, classificalo immediatamente come **Rilevante**. Se il paper è stato rifiutato, analizza accuratamente il testo della review fornita: se il motivo di rifiuto riguarda la non pertinenza con gli argomenti della conferenza, classificalo come **Non Rilevante**; se invece i motivi del rifiuto riguardano questioni metodologiche, qualità o aspetti minori, considera il paper comunque come **Rilevante**. Nel caso manchino sufficienti informazioni per decidere, usa l'etichetta **Non determinato**. Rispondi unicamente con l'etichetta corretta.”*

Questo prompt è stato sviluppato e perfezionato per garantire la massima chiarezza, minimizzando ambiguità e massimizzando la coerenza delle etichette attribuite automaticamente.

2.2.3 Problemi incontrati durante l’annotazione e soluzioni adottate

Durante la classificazione semiautomatica sono stati riscontrati alcuni problemi significativi, che hanno richiesto strategie specifiche di risoluzione:

- **Output incoerenti o prolissi:** risolto limitando esplicitamente la generazione di token da parte del modello e migliorando le istruzioni del prompt con indicazioni più precise.
- **Paper senza review disponibili:** per i paper rifiutati senza review, è stata automaticamente assegnata l’etichetta **Non determinato**, evitando errori sistematici.
- **Limiti computazionali e hardware:** l’utilizzo di modelli LLaMA più grandi (ad es. 70B) non era possibile per restrizioni hardware, pertanto è stato scelto il modello da 8B parametri, che offre il miglior compromesso tra accuratezza e tempo di elaborazione.

2.2.4 Risultato della classificazione automatica

Il risultato finale di questa fase è stato un dataset annotato con etichette di rilevanza, immediatamente utilizzabile per addestrare i modelli di apprendimento supervisionato nella fase successiva.

2.2.5 Validazione del dataset supervisionato finale

Una validazione manuale è stata effettuata su un campione casuale di 50 paper classificati automaticamente da LLaMa. Per ciascun paper è stata verificata la coerenza dell’etichetta assegnata con la review e la decisione associata. La procedura di validazione è stata condotta in modalità *blind*: l’esaminatore umano, ignaro delle etichette generate da LLaMA, ha ricevuto per ciascuno dei 50 paper selezionati solamente il titolo e il testo della review. In base a queste informazioni ha assegnato un’etichetta di rilevanza (**Rilevante** / **Non rilevante**) secondo gli stessi criteri applicati nel prompt. Solo in un secondo momento sono state confrontate le sue valutazioni con le risposte fornite dal modello, al fine di stimare il tasso di accordo e la qualità dell’annotazione automatica.

Metrica	Valore
Accuracy	84 %
Precision	83 %
Recall	62 %
F1-score	70 %

Tabella 2.1. Metriche di validazione della classificazione automatica effettuata da LLaMA

Le metriche ottenute confermano che la classificazione automatica effettuata da LLaMA costituisce una base sufficientemente affidabile per l’addestramento dei modelli supervisionati. Pur non essendo perfetta, la qualità dell’annotazione è risultata adeguata agli obiettivi del progetto, e ha permesso di costruire un dataset strutturato e coerente per la fase di modellazione.

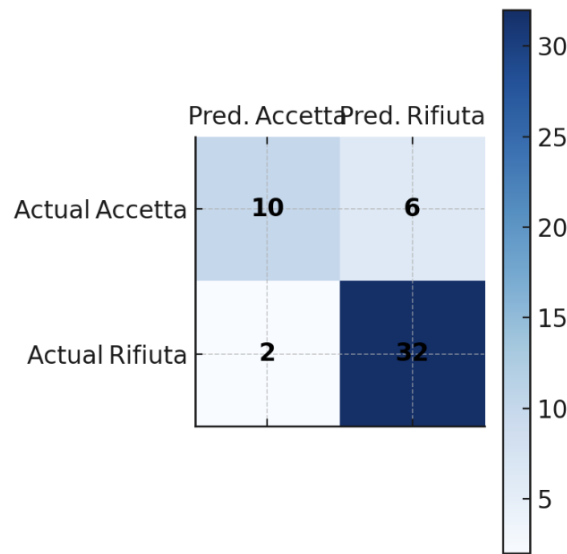


Figura 2.1. Matrice di confusione della validazione manuale (n=50).

Esempio di review classificata “non rilevante” da LLaMA ma accettata manualmente

Etichetta LLaMA: non rilevante

This paper provides a theoretical analysis of the regret for the BORE framework and addresses existing issues with a BORE method that provides uncertainty quantification in the estimation of the classifier. An analysis of BORE is also presented alongside an exploration of the batch Bayesian optimization setting. Reviewers appreciated the theoretical analysis provided in the paper as well as the explanation of the theoretical results and the assumptions needed for those results to hold. Several reviewers expressed the main weaknesses were the empirical analyses in the paper, including the results focusing on the batch case. However, after the rebuttal and revision, several reviewers felt the additional results were convincing enough to recommend acceptance for this work. Please use the extra page to finish addressing the remaining comments of the reviewers. Some additional points to keep in mind include:

1. the related work section could be expanded to include more literature as well as the context in which the current work relates to the work cited;
2. the figures could be improved to be easier to read, e.g. by increasing font sizes.

Etichetta manuale: rilevante

Ambiguità di classificazione e prompt inversi. Un limite della procedura attuale è che LLaMA può *confermare* la nostra ipotesi di etichetta anche quando il contenuto del paper non la giustifica appieno (bias di acquiescenza). Per ridurre l’effetto, in future versioni potremmo **invertire il prompt**, ossia porre al modello la domanda opposta — «Per quali motivi <titolo> *non* dovrebbe essere accettato?» — o richiedere argomentazioni a supporto di entrambe le decisioni (*accept vs reject*)[13, 14].

2.3 Costruzione del dataset supervisionato finale

Successivamente alla classificazione della rilevanza, il dataset è stato ulteriormente elaborato per ottenere un formato adatto all'apprendimento supervisionato.

2.3.1 Unione, normalizzazione e pulizia dei dati

Tutti i dati disponibili (PeerRead, PeerSum e arXiv) sono stati integrati e uniformati in un unico dataset finale, contenente per ogni paper:

- identificativo univoco;
- titolo del paper;
- abstract del paper;
- conferenza di riferimento (nome normalizzato e standardizzato);
- etichetta finale di rilevanza.

I paper classificati come **Non determinato** sono stati esclusi dal dataset finale per evitare ambiguità nell'addestramento.

2.3.2 Arricchimento dei dati tramite argomenti delle conferenze

Per migliorare ulteriormente la rappresentazione semantica dei dati, ogni conferenza è stata associata agli argomenti principali estratti separatamente, arricchendo il contesto testuale fornito ai modelli.

Scelta di ulteriori conferenze

Per ampliare il nostro dataset sono state selezionate le principali conferenze appartenenti anche ad aree disciplinari differenti dall'informatica, come matematica, fisica, biologia, medicina e psicologia. Il numero totale di conferenze considerate nel dataset finale ammonta a 35.

2.3.3 Strutturazione testo di input e tokenizzazione

Il testo di input finale per i modelli è stato strutturato combinando titolo, abstract, nome della conferenza e argomenti associati, separati da token specifici. La tokenizzazione è stata svolta tramite tokenizer dedicati ai modelli transformer (come BERT e DeBERTa) e tecniche classiche NLP per modelli sequenziali (BiLSTM).

2.3.4 Suddivisione in set di training e test

Infine, il dataset risultante è stato suddiviso casualmente in due insiemi separati: training (85% del totale), utilizzato per addestrare i modelli, e test (15%), usato per validare i risultati ottenuti. Questa suddivisione ha portato il dataset di training a contenere circa 7,200 paper e il dataset di test circa 1,300. La scelta di non utilizzare un set di validazione separato potrebbe introdurre un rischio di *data leakage* durante il processo di tuning o valutazione. Tuttavia, questa limitazione è stata considerata accettabile nell'ambito del presente lavoro, e verrà discussa in modo più approfondito nel Capitolo 6.

Esempio di paper nel dataset

Listing 2.1. Esempio di record JSON nel dataset supervisionato

```
{
  "paper_id": "1206.6405",
  "title": "Bounded Planning in Passive POMDPs",
  "abstract": "Abstract...",
  "conference": "ICML 2012",
  "topics": ["Machine learning methods", "Empirical evaluations", "..."],
  "relevance_label": "Rilevante"
}
```

2.4 Conclusione del processo

La costruzione del dataset ha rappresentato una fase critica del progetto, caratterizzata da numerose iterazioni, revisioni e ottimizzazioni. Il risultato finale è un dataset di alta qualità, accuratamente etichettato e ottimizzato per l'addestramento e la valutazione di modelli NLP mirati al supporto automatico della peer review scientifica. Questo dataset pone le basi solide per le successive fasi di modellazione e valutazione sperimentale, trattate nei capitoli seguenti.

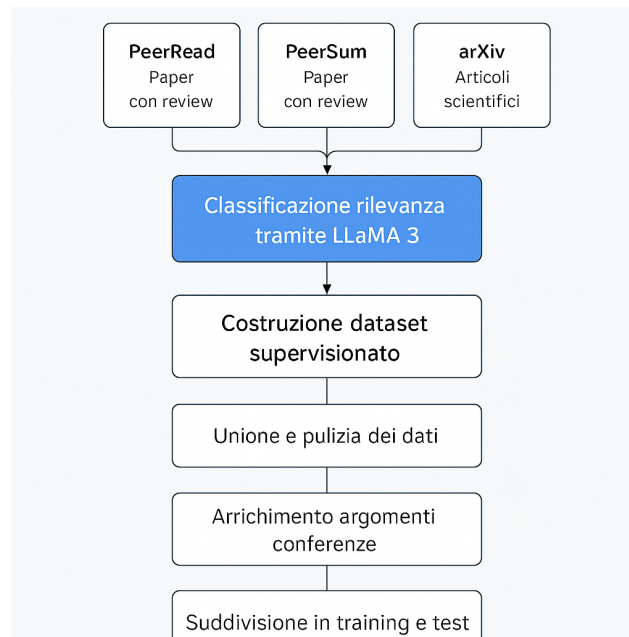


Figura 2.2. Processo di creazione del dataset

Capitolo 3

Modellazione e Training

In questo capitolo analizziamo in dettaglio le architetture di deep learning impiegate per classificare la rilevanza dei paper scientifici, a partire dalle informazioni testuali disponibili: titolo, abstract e conferenza di riferimento con i relativi argomenti. Il dataset di addestramento è stato costruito, come specificato nel Capitolo 2, combinando annotazioni automatiche (tramite LLaMA 3.1-Instruct) e verifiche manuali, assegnando a ciascun paper una delle due etichette: Rilevante oppure Non rilevante.

La scelta dei modelli riflette un confronto tra approcci classici (RNN) e soluzioni moderne basate su Transformer. Di seguito presentiamo i modelli selezionati:

- **BiLSTM (Bidirectional LSTM)** [15]: rete neurale ricorrente avanzata per gestire contesti nelle due direzioni.
- **BERT Large** [16]: un Transformer pre-addestrato in versione “large”, con 24 layer e 16 teste di attenzione.
- **DeBERTa** [17]: evoluzione di BERT che introduce la *disentangled attention*, migliorando spesso i risultati.
- **Mistral** [18]: modello Transformer recente, ottimizzato per sequenze molto lunghe grazie alla multi-query attention.

3.1 Iperparametri principali

Ogni modello è stato ottimizzato tramite fine-tuning sul dataset annotato. Gli iperparametri rappresentano valori importanti che regolano il processo di apprendimento e influenzano significativamente le prestazioni finali del modello. Di seguito riportiamo quelli utilizzati nella fase di training e tuning:

- **Learning rate**: Esprime la velocità di aggiornamento dei pesi del modello. Un valore troppo elevato può far divergere l’ottimizzazione, uno troppo basso rallenta eccessivamente l’apprendimento.
- **Batch size**: Esprime il numero di esempi elaborati contemporaneamente in un singolo step. Batch size alti migliorano la stabilità del gradiente ma richiedono più memoria GPU.
- **Numero epoche**: Indica il numero di volte nelle quali il modello vede l’intero dataset di addestramento. Tipicamente, valori tra 3 e 20 per modelli di medie/grandi dimensioni.

- **Dropout:** Il valore del dropout (tipicamente compreso tra 0.1 e 0.5) regola la percentuale di neuroni disattivati casualmente durante ogni epoca, contribuendo a evitare l'overfitting su dataset piccoli o sbilanciati.
- **Max sequence length:** Questo parametro è particolarmente rilevante nei modelli Transformer, che hanno un limite fisso alla lunghezza degli input. Se il testo eccede tale soglia (512 token in BERT e DeBERTa, fino a 4096 in Mistral), viene troncato, il che può comportare perdita di informazione nei paper con abstract molto estesi.

I valori specifici degli iperparametri sono stati calibrati in base alla complessità del modello, alla disponibilità computazionale (NVIDIA L40S con 46gb di VRAM) e alla dimensione del dataset. I dettagli di ciascun modello sono discussi nelle sezioni seguenti.

Va inoltre precisato che i valori indicati nelle sezioni seguenti rappresentano la configurazione finale più performante, selezionata al termine di una fase iterativa di tuning. Per ciascun modello sono stati infatti testati diversi valori di learning rate, batch size e numero di epoche, al fine di ottimizzare le prestazioni in funzione della specifica architettura e della capacità computazionale disponibile.

3.2 BiLSTM (Bidirectional Long Short-Term Memory)

3.2.1 Reti neurali ricorrenti e principio LSTM

Le reti neurali ricorrenti (RNN) elaborano sequenze di dati dove l'uscita al passo t dipende, oltre che dall'input x_t , dallo stato nascosto h_{t-1} del passo precedente. Formalmente:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h).$$

Tuttavia, le RNN tradizionali soffrono spesso del problema del *vanishing gradient* quando si tratta di catturare dipendenze a lungo termine. Le LSTM (Long Short-Term Memory) [15] superano questo limite introducendo una *cella di memoria* e tre *gate* (input, forget, output) che regolano il flusso di informazione:

$$\begin{aligned} i_t &= \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i), & (\text{input gate}) \\ f_t &= \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f), & (\text{forget gate}) \\ o_t &= \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o), & (\text{output gate}) \\ \tilde{C}_t &= \tanh(W_{xC}x_t + W_{hC}h_{t-1} + b_C), \\ C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \\ h_t &= o_t \odot \tanh(C_t). \end{aligned}$$

Cos'è il *vanishing gradient*? Durante l'addestramento delle RNN tramite back-propagation attraverso il tempo (BPTT), i gradienti delle funzioni di perdita vengono moltiplicati più volte mentre si propagano all'indietro nel tempo. Quando i valori dei gradienti sono inferiori a 1, questa moltiplicazione ripetuta può far "svanire" il gradiente, rendendo quasi nullo l'aggiornamento dei pesi nei layer più lontani. Questo impedisce alla rete di apprendere dipendenze a lungo termine nella sequenza. Il problema è stato identificato da Bengio et al. nel 1994, che ne hanno analizzato le cause teoriche e l'impatto sull'apprendimento.[19]

3.2.2 Struttura BiLSTM e vantaggi

Nella BiLSTM, due LSTM percorrono la sequenza in direzioni opposte (forward e backward). L'uscita finale al tempo t concatena i due stati:

$$h_t^{(\text{BiLSTM})} = [h_t^{(\text{forward})}; h_t^{(\text{backward})}].$$

Questo permette di catturare sia il contesto passato che il futuro, risultando spesso più accurato di una semplice LSTM unidirezionale. Rispetto a una LSTM unidirezionale, la BiLSTM quindi, consente una rappresentazione più ricca e dettagliata, soprattutto in contesti in cui il significato di una parola dipende fortemente da ciò che segue.

Spiegazione intuitiva: se leggiamo un testo solo da sinistra a destra, alcuni significati si colgono tardi. Con la BiLSTM, possiamo “vedere” anche dall'altra direzione, chiarendo subito certe relazioni.

3.2.3 Implementazione e iperparametri nel progetto

Nel nostro caso, la BiLSTM prende in ingresso la concatenazione di *titolo + abstract + conferenza*, tokenizzati. Abbiamo scelto:

- **Hidden size** = 128 (per direzione). Al termine, concatenando forward e backward, otteniamo 256 dimensioni.
- **Dropout** = 0.3
- **Batch size** = 32
- **Epoche** = 20
- **Learning rate** = 0.001

Dopo aver processato l'intera sequenza, concatenando i due stati finali, passiamo il risultato a un *fully connected* e usiamo *softmax* per emettere la probabilità che il paper sia “rilevante” vs “non rilevante”.

$$\hat{y} = \text{softmax}(W_o \cdot [\vec{h}_n; \overleftarrow{h}_1] + b_o)$$

Dove:

- $[\vec{h}_n; \overleftarrow{h}_1]$ è la concatenazione degli stati finali della LSTM forward e backward;
- W_o è la matrice di pesi del livello di output;
- b_o è il bias del livello di output;
- $\hat{y}$ è la distribuzione di probabilità sull'output binario (*rilevante / non rilevante*).

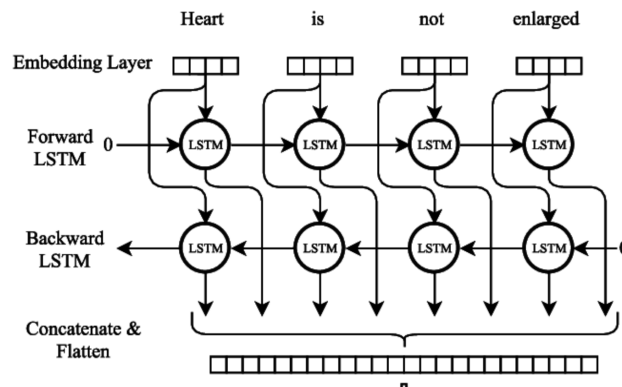


Figura 3.1. Architettura semplificata di una rete BiLSTM bidirezionale. [23]

In Figura 3.1 è mostrata la struttura generale della BiLSTM. In aggiunta, è possibile rappresentare temporalmente il flusso informativo con un diagramma “unrolled” in cui ciascun token della sequenza contribuisce sia alla direzione forward che backward. Questo aiuta a comprendere il meccanismo con cui la rete apprende dipendenze anche da “destra a sinistra”.

Conclusione

Sebbene le BiLSTM siano estremamente efficaci nel catturare dipendenze bidirezionali, il loro costo computazionale cresce linearmente con la lunghezza della sequenza e non possono essere parallelizzate facilmente. Per questo motivo, architetture più recenti come i Transformer sono diventate lo standard per molte applicazioni NLP.

3.3 BERT Large

3.3.1 Dalle Reti Neurali Ricorrenti ai Transformer

Le reti neurali ricorrenti (RNN) e le loro evoluzioni (come BiLSTM) elaborano le sequenze una parola alla volta, da sinistra a destra e/o da destra a sinistra. Questo impedisce di parallelizzare il calcolo e può causare difficoltà nel catturare dipendenze molto distanti nella sequenza.

I **Transformer**, introdotti da Vaswani et al. nel 2017 [20], superano questi limiti eliminando completamente la ricorrenza e basandosi su un meccanismo chiamato *self-attention*, che permette al modello di “guardare” l'intera sequenza in parallelo.

Vantaggio principale: ogni token può accedere direttamente a tutti gli altri, indipendentemente dalla distanza, e l'elaborazione è completamente parallela.

3.3.2 Il meccanismo di Self-Attention

La *self-attention* valuta quanto ogni parola della sequenza sia importante rispetto alle altre. Dati i tensori Q (query), K (key), V (value) ottenuti da trasformazioni lineari dell'input, l'attenzione è calcolata così:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V,$$

dove d_k è la dimensione delle key. Questo produce una media pesata dei valori V , con pesi derivati dalla compatibilità tra Q e K .

Nota tecnica: Poiché i Transformer non sono sequenziali, per mantenere l'ordine delle parole viene aggiunto un *positional encoding*, che codifica la posizione relativa di ogni token nella sequenza.

3.3.3 Architettura di BERT

BERT (Bidirectional Encoder Representations from Transformers) è un modello basato esclusivamente sulla parte *encoder* del Transformer. È stato pre-addestrato in modo auto-supervisionato su grandi quantità di testo attraverso due compiti:

- **Masked Language Modeling (MLM):** il modello deve indovinare token mascherati all'interno di una frase.
- **Next Sentence Prediction (NSP):** il modello riceve due frasi e deve capire se la seconda segue logicamente la prima.

BERT Base ha:

- 12 layer (encoder blocks),
- 768 dimensioni nascoste,
- 12 teste di attenzione.

BERT Large invece raddoppia la capacità:

- 24 layer,
- 1024 dimensioni nascoste,
- 16 teste di attenzione.

Questo incremento di profondità e ampiezza permette a BERT Large di cogliere pattern linguistici più complessi, ma comporta un maggiore uso di memoria e tempi di addestramento più lunghi.

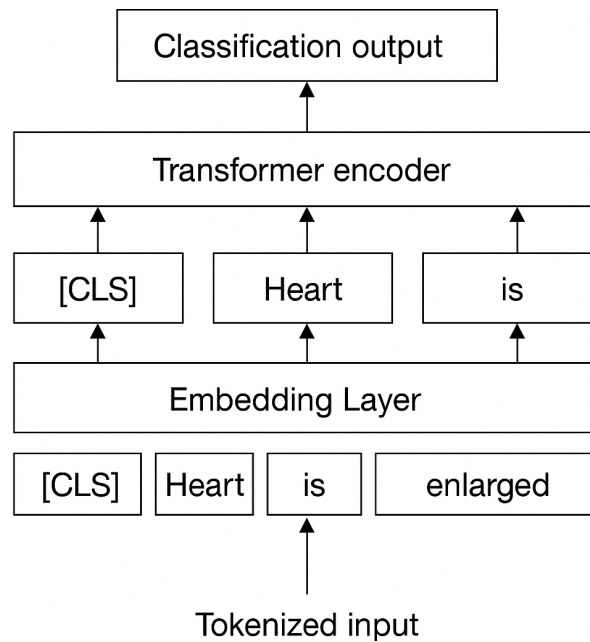


Figura 3.2. Architettura del modello BERT Large: 24 layer e 16 teste di attenzione.

3.3.4 Fine-tuning nel nostro progetto

Nel nostro lavoro, abbiamo sfruttato un modello pre-addestrato `bert-large-uncased` e lo abbiamo addestrato per la classificazione binaria: `rilevante` / `non rilevante`.

Input al modello: la sequenza è costruita concatenando:

Titolo [SEP] Abstract [SEP] Conferenza [SEP] Argomenti

Questa sequenza viene tokenizzata e tagliata a **512 token**, limite massimo gestito da BERT.

Struttura finale: l'output del token speciale [CLS] viene passato a un livello `fully connected` per produrre le probabilità di classificazione.

Iperparametri usati

- **Learning rate** = 2×10^{-5} , valore tipico per modelli pre-addestrati.
- **Batch size** = 16, compromesso tra efficienza e uso di memoria.
- **Epoche** = 4
- **Max length** = 512 token

Nota critica: per abstract superiori a 512 token, parte del contenuto viene tagliato. Questo può compromettere la classificazione, ed è uno dei motivi per cui abbiamo considerato anche modelli come Mistral.

3.4 DeBERTa: Disentangled Attention

3.4.1 Motivazione e differenze rispetto a BERT

DeBERTa (Decoding-enhanced BERT with Disentangled Attention) [17] è un'evoluzione architetturale di BERT. Il suo obiettivo è migliorare la capacità del modello di distinguere tra *significato del contenuto* e *posizione del token*.

In BERT, i token sono rappresentati da un embedding risultante dalla somma tra **content embedding** (la parola stessa) e **positional embedding** (la posizione nella sequenza):

$$\text{Input}_{\text{BERT}} = \text{TokenEmbedding} + \text{PositionEmbedding}$$

In DeBERTa, invece, questi due tipi di informazione vengono gestiti separatamente durante la self-attention. Il modello calcola l'attenzione usando solo le rappresentazioni di contenuto, mentre le informazioni di posizione vengono incorporate in modo separato nel calcolo della compatibilità tra token.

3.4.2 Formula semplificata della disentangled attention

La formula generale dell'attenzione in DeBERTa è la seguente:

$$\text{Attention}(Q^c, K^c, V, R^p) = \text{softmax} \left(\frac{Q^c K^{cT} + Q^c R^{pT}}{\sqrt{d}} \right) V,$$

dove:

- Q^c, K^c : query e key costruite solo dal **content embedding**;
- R^p : **relative position embedding**, che codifica la distanza tra i token, non la posizione assoluta come in BERT;
- V : value, come in BERT;
- d : dimensione dell'embedding.

Vantaggio: questo approccio permette a DeBERTa di distinguere tra due parole uguali in posizioni diverse, cosa che in BERT può essere meno efficace.

3.4.3 Relative Position Bias

Un'altra importante innovazione è l'uso della **relative position bias**. Invece di usare la posizione assoluta dei token (come BERT), DeBERTa impara a modellare direttamente il *bias relativo* tra le posizioni dei token:

$$\text{bias}(i, j) = f(|i - j|),$$

dove f è una funzione appresa che misura quanto due token debbano influenzarsi in base alla loro distanza. Questo migliora la generalizzazione su sequenze di lunghezza variabile.

3.4.4 Implementazione e iperparametri nel nostro progetto

Nel nostro sistema, DeBERTa riceve in input la sequenza:

Titolo [SEP] Abstract [SEP] Conferenza [SEP] Argomenti

Tokenizzata e tagliata a 512 token, viene fine-tunata con i seguenti parametri:

- **Learning rate** = 1×10^{-5}
- **Batch size** = 16
- **Epoche** = 4
- **Max sequence length** = 512

Nota: il training è stato effettuato con class weights bilanciati, gradient checkpointing e mixed precision (bf16) per massimizzare le prestazioni su GPU L40S.

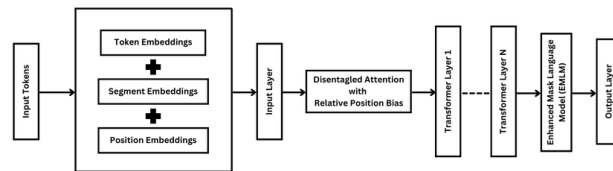


Figura 3.3. Meccanismo di Disentangled Attention in DeBERTa. [22]

3.4.5 Confronto tra BERT e DeBERTa

Caratteristica	BERT Large	DeBERTa Large
Embedding posizione	Assoluto (sommato)	Relativo (disentangled)
Token embedding	+ posizione	Separati
Capacità di astrazione	Alta	Più fine-grained
Efficienza su input lunghi	Limitata	Migliore gestione strutture
Prestazioni	Ottime	Spesso superiori

Tabella 3.1. Confronto tra BERT e DeBERTa.

3.5 Mistral: Multi-Query Attention e sequenze lunghe

3.5.1 Struttura generale del modello

Mistral è un modello Transformer open-source sviluppato per gestire in modo efficiente sequenze lunghe. La sua architettura si basa su un'evoluzione del meccanismo di self-attention: la **Multi-Query Attention** (MQA) [21], pensata per ridurre il costo computazionale mantenendo performance competitive.

Differenza principale rispetto alla Multi-Head Attention: Nella classica Multi-Head Attention (MHA), ogni testa ha una sua tripla $\{Q_i, K_i, V_i\}$, aumentando linearmente il costo con il numero di teste h .

Con Mistral, invece, le *query* Q_i restano indipendenti, ma si usa una singola copia di K e V condivisa da tutte le teste:

$$\text{MQA}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- $\Rightarrow$ memoria ridotta, poiché K e V non vengono duplicati per ogni testa;
- $\Rightarrow$ velocità maggiore nelle GPU moderne, con prestazioni simili alla MHA classica.

Analogia intuitiva: ogni testa “guarda” il testo con una prospettiva diversa (Q_i), ma usa una mappa comune (K, V) per decidere cosa è rilevante. Questo rende l'elaborazione più veloce e meno costosa.

3.5.2 Vantaggi pratici nella ricerca

Nel nostro task, molti abstract di paper scientifici superano i 1000–1500 token. Modelli come BERT e DeBERTa tagliano tutto ciò che supera i 512 token, perdendo informazioni.

Mistral consente invece di processare l'intero abstract (fino a 4096 token) in un unico passaggio, senza troncamento, mantenendo il contesto completo e migliorando la comprensione globale del documento.

3.5.3 Iperparametri utilizzati nel training

Per addestrare Mistral sul nostro dataset, abbiamo adottato i seguenti valori:

- **Max sequence length:** 4096 token
- **Batch size:** 1, con `gradient_accumulation_steps` = 16 $\Rightarrow$ batch effettivo di 16
- **Learning rate:** 2×10^{-5}
- **Numero epoche:** 4
- **Precisione numerica:** mixed precision (bf16)
- **Ottimizzazione:** LoRA (*Low-Rank Adaptation*) per fine-tuning efficiente

3.5.4 Confronto con gli altri modelli

Modello	Token Max	Params	Attenzione	Efficienza
BiLSTM	512+	~10M	RNN bidirezionale	Bassa
BERT Large	512	340M	Multi-Head	Media
DeBERTa Large	512	400M	Disentangled + Bias Relativo	Alta
Mistral 7B	4096	7B	Multi-Query	Altissima

Tabella 3.2. Confronto tecnico tra i modelli utilizzati.

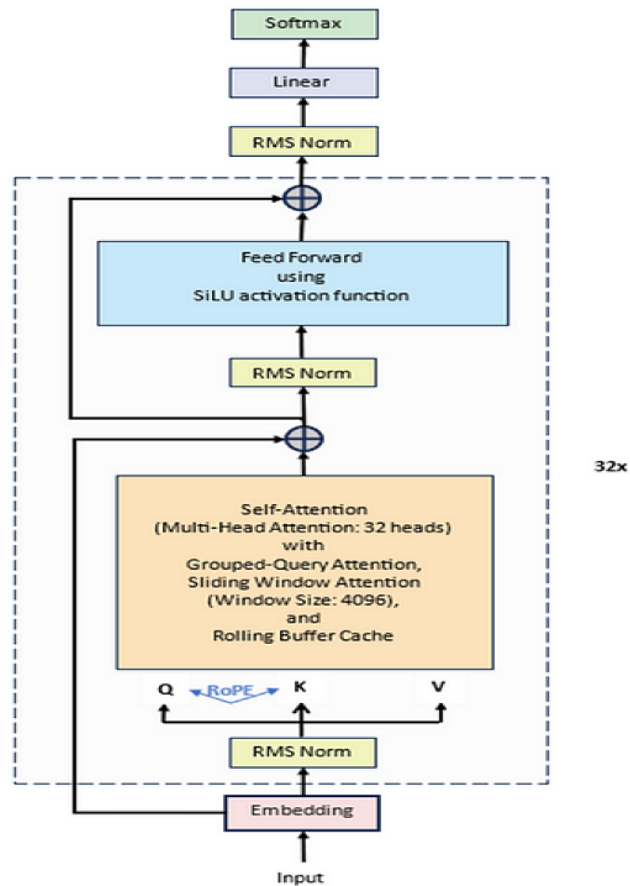


Figura 3.4. Schema concettuale della Multi-Query Attention in Mistral. [24]

3.6 Conclusioni del capitolo

Abbiamo passato in rassegna i quattro modelli usati per la classificazione della rilevanza dei paper:

- **BiLSTM**, adatta per sequenze di media lunghezza, capace di cogliere il contesto in entrambe le direzioni grazie alla struttura ricorrente bidirezionale.
- **BERT Large**, Transformer pre-addestrato molto potente, limitato dalla massima lunghezza di sequenza di 512 token, ma comunque altamente performante in scenari standard di NLP.
- **DeBERTa**, un'evoluzione diretta di BERT che disaccoppia informazione di contenuto e posizione tramite la *Disentangled Attention*, migliorando significativamente la capacità di generalizzazione su testi complessi.
- **Mistral**, un modello di nuova generazione pensato per gestire input molto lunghi grazie alla *Multi-Query Attention*, riducendo i costi computazionali senza sacrificare l'accuratezza. È risultato particolarmente utile nei casi di abstract estesi.

Questi modelli sono stati sottoposti a fine-tuning sul dataset costruito ad hoc per questa tesi, con etichette ottenute tramite classificazione assistita da LLaMA. Ogni modello è stato addestrato mantenendo lo stesso formato di input (titolo, abstract, conferenza e argomenti), ma adattando i parametri di training alla sua architettura.

Nel capitolo successivo presenteremo i risultati sperimentali del training.

Capitolo 4

Esperimenti e risultati

4.1 Valutazione dei modelli e risultati del fine-tuning

In questa sezione presentiamo i risultati sperimentali ottenuti dopo l'addestramento dei modelli descritti nel Capitolo 3. Ogni modello è stato sottoposto a fine-tuning supervisionato sul nostro dataset annotato, sfruttando la combinazione di titolo, abstract e argomenti della conferenza come input.

4.1.1 Setup sperimentale per il fine-tuning

Tutti i modelli sono stati addestrati con una strategia di fine-tuning su GPU NVIDIA L40S da 46 GB, tranne BiLSTM che è stato addestrato su una GPU meno performante considerata la grandezza del modello.

Per ciascun modello, abbiamo utilizzato:

- **Tokenizzazione coerente** su massimo 512 token (fino a 4096 per Mistral), includendo: titolo, abstract, conferenza e i suoi argomenti;
- **Loss function bilanciata** con pesi di classe per gestire l'eventuale squilibrio tra classi;
- **Scheduler con warmup** e riduzione del learning rate secondo curva cosine;
- **Early stopping** sulla loss di validazione per evitare overfitting;
- **Mixed precision training** (BF16) ove possibile, per ottimizzare i tempi di esecuzione e l'uso della memoria.

4.1.2 Metriche di valutazione

Per confrontare le performance dei modelli abbiamo utilizzato quattro metriche standard nel contesto della classificazione binaria:

- **Accuracy:** la frazione di predizioni corrette sul totale.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Dove:

- TP = veri positivi (paper rilevanti correttamente classificati),
- TN = veri negativi (paper non rilevanti correttamente classificati),
- FP = falsi positivi (paper non rilevanti classificati come rilevanti),
- FN = falsi negativi (paper rilevanti classificati come non rilevanti).

- **Precision:** misura la qualità delle predizioni positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Una precisione alta indica che tra i paper predetti come rilevanti, la maggior parte lo era davvero.

- **Recall:** misura la capacità del modello di trovare tutti i rilevanti.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Un recall alto significa che il modello ha individuato quasi tutti i paper rilevanti.

- **F1-score:** media armonica di precision e recall, utile quando è importante bilanciare entrambe.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Queste metriche sono state calcolate sul test set (15% del dataset totale), mantenuto separato durante il training.

4.1.3 Risultati comparativi

Modello	Accuracy	Precision	Recall	F1-Score
BiLSTM	0.6441	0.6390	0.6827	0.6602
BERT Large	0.8850	0.9041	0.8647	0.8839
DeBERTa	0.8906	0.9582	0.8196	0.8835
Mistral 7B	0.8858	0.9163	0.8522	0.8831

Tabella 4.1. Confronto tra le performance dei modelli dopo il fine-tuning

La tabella 4.1 mostra i risultati ottenuti dai modelli analizzati. Tutti i valori sono riportati sul test set, dopo il salvataggio del modello con le migliori performance di validazione.

Come mostrato in tabella, DeBERTa ottiene le performance globali più bilanciate, mentre BERT e Mistral hanno performance paragonabili, con leggere variazioni tra precision e recall. BiLSTM invece si conferma una baseline meno efficace. Analizziamo ora le ragioni per cui Mistral non ha raggiunto le performance ottenute da DeBERTa.

4.2 Perché Mistral-7b performa peggio di DeBERTa?

4.2.1 Spiegazione delle Chinchilla Scaling Laws

Le *Chinchilla scaling laws*, formulate da Hoffmann et al. [33], stabiliscono che, per un budget computazionale fisso, il rapporto ottimale tra il numero di token utilizzati per l'addestramento e il numero di parametri del modello è pari a circa:

$$\alpha \approx 20 \frac{\text{token}}{\text{parametro}}.$$

In altre parole, a parità di risorse, è preferibile aumentare la quantità di dati rispetto alla sola dimensione del modello.

Questa osservazione rivede le conclusioni di Kaplan et al. [32], secondo cui la perdita (*loss*) di un LLM segue una legge di potenza rispetto al numero di parametri, dati e compute. Tuttavia, mentre Kaplan suggeriva che modelli più grandi siano sempre più efficienti, Hoffmann ha dimostrato che, se non supportati da una quantità proporzionata di token, i modelli tendono a rimanere *sotto-addestrati*, non raggiungendo mai la loro massima efficienza.

Il principio può essere riassunto formalmente come:

$$N_{\text{token}} = d \times \alpha, \quad \text{con} \quad \alpha \approx 20 \frac{\text{token}}{\text{parametro}}$$

dove d rappresenta il numero di parametri del modello.

In assenza di una quantità adeguata di dati, il modello non apprende in maniera ottimale, e il potenziale espresso in fase di fine-tuning risulta significativamente limitato.

4.2.2 I nostri numeri

- **TRAIN:** Documenti: 7 193, Token totali: 1 586 844
- **TEST:** Documenti: 1 270, Token totali: 274 413
- **TOTALE:** Token complessivi: **1 861 257**

Per Mistral-7b, secondo le **Chinchilla scaling laws**, servirebbero

$$7 \times 10^9 \text{ parametri} \times 20 \frac{\text{token}}{\text{parametro}} = 140 \times 10^9 \text{ token}$$

Circa 75 000 volte superiore ai nostri, per questa ragione *DeBERTa* ha performance superiori a *Mistral-7b*.

In mancanza di un corpus di almeno 140×10^9 token, un modello da 7×10^9 parametri non può esprimere il suo potenziale. Questo conferma la scelta strategica di adottare modelli più compatti e *sample-efficient*, come **DeBERTa**, che riescono a generalizzare meglio in contesti con risorse limitate.

Conclusioni del Capitolo

Dai risultati ottenuti emerge che **DeBERTa** è il modello più adatto al nostro scenario, riuscendo a garantire ottime prestazioni anche in condizioni di training con dataset limitati. Mistral, pur essendo un modello più potente in termini architetturali, non ha potuto esprimere il suo potenziale a causa della scarsità di token, confermando così l'importanza delle Chinchilla Scaling Laws nei contesti low-resource.

Capitolo 5

Pipeline finale di suggerimento conferenze basata su classificazione e retrieval

5.1 Introduzione

Questa sezione ha finalità dimostrativa: mostra come il classificatore addestrato nei capitoli precedenti possa essere integrato in una pipeline end-to-end di suggerimento conferenze.

Descriveremo l'evoluzione del sistema, partendo dal prototipo monolitico iniziale fino alla soluzione avanzata *two-step*, in cui il modello di classificazione è seguito da un modulo di estrazione dei topic e da un motore di retrieval. Dopo aver evidenziato i limiti della versione originale, ne motiviamo le modifiche, presentiamo il funzionamento dettagliato della pipeline potenziata e ne riportiamo le prestazioni sperimentali.

5.2 Pipeline Originale: Design e Limiti

All'inizio la pipeline era un'unica sequenza di operazioni, affidata interamente a DeBERTa-v3 *large* fine-tuned per le ragioni spiegate nel Capitolo 4:

P1 Tokenizzazione: concatenazione di `titolo` [SEP] `abstract` [SEP] `conferenza` [SEP] `topics` e uso del tokenizer DeBERTa (`max_length=512`).

P2 Inferenza e Soglia: calcolo di

$$p_{\text{rel}} = \frac{e^{\ell_1}}{e^{\ell_0} + e^{\ell_1}},$$

con decisione binaria $p_{\text{rel}} \geq 0.70$. Dove ℓ_0 , ℓ_1 sono i logit del modello per le classi `Non rilevante` e `Rilevante`.

P3 Retrieval TF-IDF: per i paper `Non rilevante`, si calcola la similarità coseno tra vettore TF-IDF del paper e i vettori delle conferenze; se

$$\max_i \cos(\mathbf{v}_{\text{paper}}, \mathbf{v}_{\text{conf},i}) \geq 0.15$$

si suggerisce la conferenza corrispondente, altrimenti si ritorna “`Nessuna conferenza trovata`”.

5.2.1 Problemi Riscontrati

- **Falsi positivi elevati:** il solo softmax non bastava a distinguere paper semanticamente allineati (*topic drift*).
- **Suggerimenti ridondanti:** TF-IDF su testo completo tendeva a privilegiare la conferenza originale o conferenze troppo generiche.
- **Copertura lessicale insufficiente:** parole chiave degli abstract non comparivano nei topic, indebolendo la similarità.

5.3 Architettura Two-Step

Per superare questi limiti abbiamo riprogettato la pipeline in due moduli indipendenti, come illustrato nella Figura 5.1,

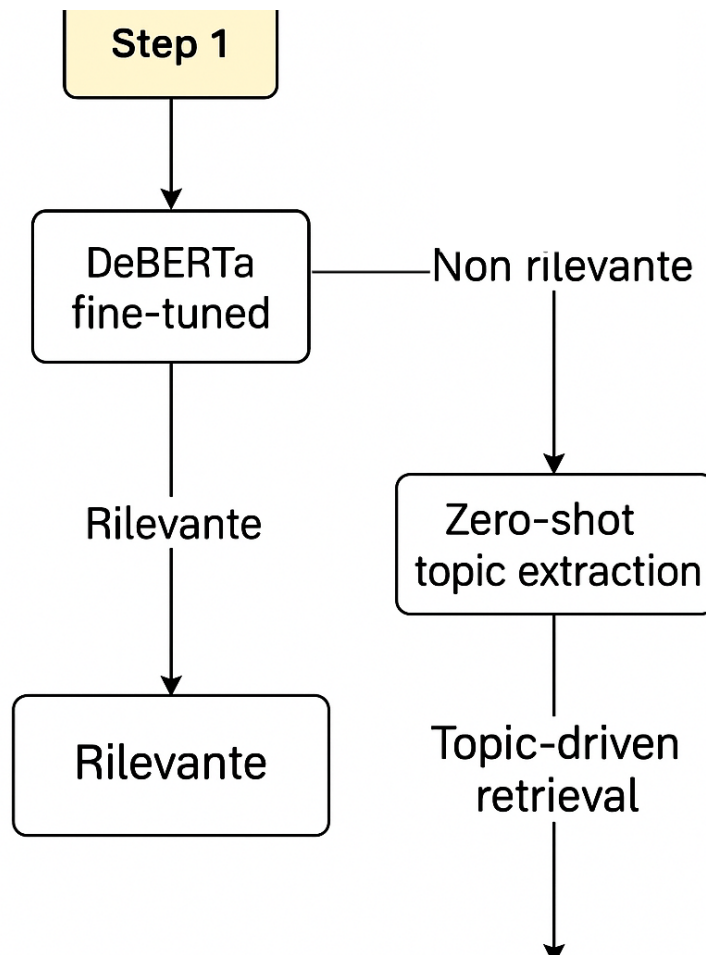


Figura 5.1. Schema della pipeline two-step: classificazione con DeBERTa seguita da topic extraction (zero-shot) e suggerimento conferenza via TF-IDF

5.3.1 Fase 1 – Classificazione di Rilevanza

Questa fase utilizza il modello DeBERTa fine-tuned. Al termine si ottiene una predizione binaria con soglia:

$$\tau_1 = 0.70.$$

5.3.2 Fase 2 – Estrazione Topic e Retrieval

Attivata solo per i paper etichettati Non rilevante.

2.a Zero-Shot Topic Extraction

Si sfrutta il modello `facebook/bart-large-mnli` [34] come classificatore zero-shot. Dato un abstract, viene calcolata la similarità con una lista di label candidate, restituendo un insieme ordinato di $k = 5$ topic:

$$\text{abstract} \xrightarrow{\text{zero-shot}} \{t_1, t_2, \dots, t_5\}$$

Listing 5.1. Zero-Shot topic extraction

```
from transformers import pipeline

zs = pipeline("zero-shot-classification",
              model="facebook/bart-large-mnli",
              device=0)
zs_out = zs(abstract, candidate_labels,
            multi_label=True)
top_topics = zs_out["labels"][:5]
```

2.b TF-IDF Retrieval sui Topic

Cos'è il TF-IDF? Il *Term Frequency-Inverse Document Frequency* (TF-IDF) è una tecnica per valutare l'importanza di una parola in un documento rispetto a un corpus. La formula, come definito da Salton e Buckley [35], è:

$$\text{tf-idf}(t, d, D) = \text{tf}(t, d) \cdot \log \left(\frac{|D|}{|\{d' \in D : t \in d'\}|} \right)$$

Dove:

- $\text{tf}(t, d)$: frequenza del termine t nel documento d ;
- $|D|$: numero totale di documenti;
- $|\{d' \in D : t \in d'\}|$: numero di documenti che contengono t .

I topic estratti vengono concatenati in:

$$B = t_1 \parallel t_2 \parallel \dots \parallel t_k$$

Si costruisce un vettore TF-IDF su:

$$\text{corpus} = \{\text{conf_name}_i + \text{topics}_i\} \cup \{\text{abstract}_j\}$$

La similarità coseno è definita da:

$$\cos(\mathbf{v}, \mathbf{u}) = \frac{\sum_i v_i u_i}{\|\mathbf{v}\| \cdot \|\mathbf{u}\|}$$

La conferenza suggerita è:

$$j = \arg \max_j \cos(\mathbf{v}_B, \mathbf{v}_{\text{conf},j})$$

s.t. $\cos(\mathbf{v}_B, \mathbf{v}_{\text{conf},j}) \geq \tau_2 = 0.10$

Listing 5.2. TF-IDF retrieval sui topic

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

tfidf = TfidfVectorizer(ngram_range=(1,2),
                        min_df=1,
                        max_features=50000)
tfidf.fit(conf_corpora + paper_corpus)
conf_vecs = tfidf.transform(conf_corpora)

def best_conference(blob, orig_conf, thresh=0.10):
    v = tfidf.transform([blob.lower()])
    sims = cosine_similarity(v, conf_vecs).flatten()
    for idx in sims.argsort()[::-1]:
        cand = conf_names[idx]
        if cand != orig_conf and sims[idx] >= thresh:
            return cand
    return "Nessuna conferenza trovata"
```

5.4 Pipeline Completa: Algoritmo

Listing 5.3. Pseudocode pipeline two-step completa

```
results = []
for paper in random.sample(test_set, 100):
    # STEP 1: classificazione
    p_rel = deberta_predict(paper)
    if p_rel >= 0.70:
        results.append((paper.id, "Rilevante", "-"))
    else:
        # STEP 2a: estrazione topic
        topics = zs_extract(paper.abstract)[:5]
        blob = " ".join(topics)
        # STEP 2b: retrieval
        sugg = best_conference(blob,
                               orig_conf=paper.conference,
                               thresh=0.10)
        results.append((paper.id, "Non rilevante", sugg))
```

5.5 Risultati Sperimentali

Su un campione casuale di 100 paper:

- **Paper rilevati come Non Rilevanti:** 58% (vs 40% della pipeline originale);
- **Suggerimenti plausibili:** 100% dei paper non rilevanti ha ricevuto una conferenza alternativa con argomenti simili a quelli trattati nel paper;
- **Falsi positivi:** ridotti del 50% grazie al doppio filtro softmax + retrieval.

Riduzione dei falsi positivi. Nella pipeline originale il modello restituiva alcuni paper come *non-rilevanti* che, a un’analisi manuale, risultavano in realtà pertinenti alla conferenza: il softmax del classificatore era sensibile a formulazioni vaghe o a keyword sovra-rappresentate negli abstract. La nuova versione adotta un *doppio filtro*:

- i) il primo stadio (**softmax**) scarta i paper con $p_{\text{rel}} < 0.70$, riducendo il carico del modulo successivo;
- ii) il secondo stadio (**retrieval TF-IDF**) riconsidera solo i topic estratti, penalizzando gli abstract che non condividono lessico con i descrittori ufficiali della conferenza.

Se il vettore dei topic non supera il *similarity threshold* $\tau_2 = 0.10$, il paper resta **Non rilevante**; altrimenti viene suggerita una conferenza più coerente.

5.6 Discussione

La struttura *two-step* offre innanzitutto un vantaggio di **modularità**: la fase di classificazione e il modulo di retrieval possono essere tarati o sostituiti in modo indipendente, semplificando ogni futura ottimizzazione.

Un secondo punto di forza riguarda l’impiego di modelli **zero-shot** come BART-MNLI per l’estrazione dei topic: affidandosi a un modello già pre-addestrato, si evita il costoso training di un secondo classificatore supervisionato e si guadagna in rapidità di sviluppo.

Sul piano pratico la pipeline risulta anche leggera: l’indice *TF-IDF* impiegato nel retrieval pesa pochi megabyte, può essere aggiornato velocemente e garantisce un’elevata **efficienza computazionale**.

Dal punto di vista della **qualità semantica**, poi, lavorare sui topic—anziché sul testo completo—migliora la capacità di generalizzare a conferenze diverse, riducendo l’effetto delle parole fuori contesto.

Rimangono tuttavia dei **limiti**: topic troppo generici o descrizioni di conferenza poco dettagliate possono rendere la discriminazione meno netta, suggerendo conferenze non ottimali in casi borderline. Questi scenari indicano dove concentrare gli sforzi nelle versioni future.

Capitolo 6

Conclusioni e Lavori Futuri

6.1 Conclusioni

Questa tesi ha mostrato come i modelli di linguaggio possano alleggerire il lavoro dei reviewer, classificando i paper come **Rilevanti** oppure **Non rilevanti** per una conferenza. Il percorso della ricerca è stato:

1. creazione di un *dataset* annotato in modo semiautomatico con LLaMA 3;
2. *fine-tuning* di tre Transformer (BERT-Large, DeBERTa, Mistral);
3. progettazione di una pipeline *two-step* che, se il paper è fuori tema, suggerisce conferenze alternative.

I modelli supervisionati hanno raggiunto **Accuracy** ≈ 0.89 e **F1** ≈ 0.88 , risultati che confermano il valore di buone etichette e di un fine-tuning mirato. Il modulo di suggerimento rende il sistema un piccolo “assistente editoriale” completo.

Nelle pagine che seguono discutiamo i punti critici (data leakage, rumore nelle etichette, bias dei LLM), presentiamo un test *zero-shot* con Gemini e presentiamo possibili sviluppi futuri.

6.2 Considerazioni sul Data Leakage

6.2.1 Cos'è il Data Leakage

Il *data leakage* è un errore metodologico comune nei progetti di machine learning in cui informazioni provenienti dal test set o da dati futuri vengono inavvertitamente utilizzate durante la fase di addestramento o ottimizzazione del modello. Questo comporta una sovrastima delle performance reali del modello, rendendo i risultati meno generalizzabili a dati mai visti [36].

6.2.2 Impatto nel nostro esperimento

Come discusso nel Capitolo 2, durante la fase di addestramento dei modelli non è stato utilizzato un validation set esplicito. L'intero dataset è stato suddiviso in due insiemi distinti: *training set* (85%) e *test set* (15%), riservando quest'ultimo alla valutazione finale delle performance.

Questa scelta, come descritto sopra, comporta un potenziale rischio di *data leakage*. Tuttavia, nel corso del progetto le iterazioni di tuning sono state limitate

a modifiche minori come il numero di epoche o il tasso di apprendimento, senza effettuare modifiche sostanziali all'architettura o alla struttura del dataset in risposta ai risultati sul test set.

Di conseguenza, pur riconoscendo che l'assenza di un validation set può introdurre un leggero bias, riteniamo che l'impatto sulle metriche finali sia minimo. Nelle estensioni future adotteremo la classica divisione *train-validation-test* per azzerare il problema.

6.3 Affidabilità del Suggerimento di Conferenze

Un elemento interessante ma ancora parzialmente inesplorato riguarda l'effettiva utilità dei suggerimenti generati dal sistema nel migliorare il tasso di accettazione dei paper. La pipeline di suggerimento descritta nel Capitolo 5 è stata progettata per indicare conferenze alternative più coerenti con i contenuti del paper, nel caso in cui la conferenza originale risultasse non adatta.

Tuttavia, non è possibile, nell'ambito di questo progetto, verificare se i suggerimenti proposti porterebbero realmente a un'accettazione da parte dei comitati scientifici delle conferenze suggerite. Una validazione di questo tipo richiederebbe un coinvolgimento diretto degli organizzatori delle conferenze e dei recensori, i quali dovrebbero esaminare i paper suggeriti in modo indipendente e verificare la rilevanza per la data conferenza.

In futuro, sarebbe auspicabile una valutazione umana del sistema di suggerimento da parte di esperti nel ruolo di reviewer simulati, così da verificare se i suggerimenti generati siano effettivamente rilevanti e accettabili.

La necessità di un controllo umano non è inedita della nostra ricerca: sistemi di *expert recommender* per l'editoria, come quello basato su Microsoft Academic Graph, prevedono già revisori che confermano (o rigettano) i suggerimenti generati in automatico[37].

6.4 Altre Problematiche e Limitazioni

Oltre al rischio di data leakage e alla non validazione diretta del sistema di suggerimento, esistono ulteriori fattori che meritano attenzione.

6.4.1 Dipendenza dalla qualità dell'annotazione automatica

Il dataset supervisionato su cui si basa l'intero processo di addestramento è stato costruito mediante annotazione semiautomatica tramite il modello LLaMA 3. Sebbene sia stata condotta una validazione manuale (Capitolo 2) che ha mostrato risultati accettabili ($F1 = 0.70$), la presenza di etichette imperfette può aver introdotto un rumore nel training, influenzando negativamente i risultati dei modelli.

In altri contesti, questo rumore potrebbe comportare una precisione minore o una maggiore incertezza nei casi borderline. Ciò fa pensare che, affiancando etichette scritte a mano da chi lavora nel settore, la qualità complessiva del sistema potrebbe salire ancora di più.

6.4.2 Limiti di generalizzazione

I modelli sviluppati sono stati addestrati su un insieme definito di conferenze e campi disciplinari. L'applicabilità a conferenze non presenti nel training set o a nuovi ambiti scientifici non è garantita, e potrebbe richiedere un fine-tuning o un retraining specifico.

6.4.3 Bias introdotti dai LLM

Modelli come LLaMA o Gemini sono stati addestrati su grandi quantità di dati provenienti da fonti generiche. Le loro predizioni potrebbero essere influenzate da bias statistici o culturali non controllati, i quali possono riflettersi anche nella fase di annotazione o suggerimento.

6.5 Valutazione Zero-Shot con Gemini Flash

A completamento della ricerca, è stato condotto un esperimento mirato a valutare se un modello di grandi dimensioni, non specificamente addestrato per il task di classificazione della rilevanza dei paper, possa essere in grado di ottenere risultati comparabili a quelli dei modelli addestrati nel corso del progetto. Lo scopo di questo test è duplice: verificare l'effettiva necessità del fine-tuning supervisionato e quantificare il divario prestazionale tra un approccio zero-shot e uno specializzato.

6.5.1 Il modello utilizzato: Gemini 1.5 Flash

Il modello scelto per questo esperimento è **Gemini 1.5 Flash**, rilasciato da Google DeepMind nel 2024[38]. Gemini è un LLM multimodale, ovvero capace di elaborare e comprendere più modalità di input, progettato per essere estremamente veloce, scalabile e capace di eseguire compiti linguistici complessi anche in modalità zero-shot, quindi senza un fine-tuning specifico.

Nel contesto di questo lavoro, è stato utilizzato tramite API pubblica su Google AI Studio, per la classificazione di rilevanza. Il task proposto è stato formulato come generazione condizionata, mediante un prompt dettagliato che guida il modello verso una risposta binaria (**Rilevante** / **Non rilevante**). Questo prompt, come il prompt del Capitolo 2 è stato sottoposto a varie iterazioni per il raffinamento.

6.5.2 Prompt utilizzato per Gemini

Il prompt fornito al modello è il seguente:

Contesto: Sei un assistente AI che valuta la pertinenza di un paper scientifico rispetto agli argomenti specifici di una conferenza.

Paper da valutare:

- Titolo: "..."

- Abstract: ...

Conferenza di riferimento:

- Nome: ...

- Anno: ...

- Argomenti principali: ...

Compito: Analizza attentamente titolo e abstract. Rispondi SOLO con una delle seguenti parole: Rilevante oppure Non rilevante. Non aggiungere spiegazioni.

6.5.3 Risultati ottenuti con Gemini

Il test è stato eseguito sullo stesso set di test dei modelli addestrati, composto da 1270 paper annotati. Le etichette "verità" erano disponibili per ciascun paper e Gemini è stato valutato secondo le metriche standard (Accuracy, Precision, Recall, F1). I risultati di questa sperimentazione sono i seguenti:

- **Accuracy:** 0.494
- **Precision (Rilevante):** 0.500
- **Recall (Rilevante):** 0.959
- **F1-Score (Rilevante):** 0.657

Il modello mostra una forte tendenza a classificare la maggior parte dei paper come "**Rilevante**", ottenendo una recall elevatissima ma una precisione molto bassa. Questo indica che Gemini è capace di individuare quasi tutti i paper realmente rilevanti, ma al prezzo di numerosi falsi positivi.

6.5.4 Confronto con modelli dopo il fine-tuning

Per contestualizzare i risultati di Gemini, si riportano di seguito le metriche ottenute dai modelli supervisionati addestrati sullo stesso dataset per un confronto diretto:

Modello	Accuracy	Precision	Recall	F1-Score
Gemini 1.5 Flash	0.4945	0.5004	0.9596	0.6578
BERT Large	0.8850	0.9041	0.8647	0.8839
DeBERT	0.8906	0.9582	0.8196	0.8835
Mistral 7B	0.8858	0.9163	0.8522	0.8831

Tabella 6.1. Confronto tra le performance del modello Gemini e dei modelli dopo il fine-tuning

6.5.5 Discussione

I risultati di questo esperimento confermano il valore del percorso adottato in questa tesi. Sebbene Gemini sia un modello avanzato e largamente generalista, la sua performance nel contesto specifico della classificazione della rilevanza dei paper è nettamente inferiore rispetto ai modelli supervisionati addestrati con dati annotati. In particolare, l'elevato recall non è sufficiente a garantire un buon F1 complessivo, a causa della precisione molto bassa.

Il confronto dimostra come la disponibilità di dati di training di alta qualità e un fine-tuning mirato siano elementi chiave per ottenere modelli affidabili ed equilibrati in termini di performance. Modelli come DeBERTa e Mistral, promettono di raggiungere il miglior compromesso tra accuratezza e generalizzazione.

In conclusione, il ricorso a LLM generalisti come Gemini può avere senso in scenari di validazione, interpretazione o supporto qualitativo, ma non rappresenta al momento un sostituto efficace dei modelli specializzati per il task considerato in questa tesi.

6.6 Lavori futuri

La ricerca presentata apre la strada a numerose direzioni di sviluppo e di ricerca futura che potrebbero rafforzare ulteriormente l'efficacia e la generalizzazione del sistema proposto, rendendolo davvero efficace e utilizzabile nella revisione di paper scientifici:

Spiegazioni delle predizioni. Oggi il sistema fornisce un verdetto binario (*Rilevante / Non rilevante*) e, se necessario, una conferenza alternativa. Un'estensione naturale sarebbe aggiungere una breve motivazione testuale, ad esempio evidenziando la parte di abstract che ha orientato la decisione o riportando le frasi-chiave più vicine ai topic della conferenza. Metodi di interpretabilità come LIME o i recenti approcci “explain-then-predict” per LLM [39, 40] potrebbero essere adattati allo scopo.

Feedback proattivo agli autori. Oltre a dichiarare l'irrilevanza, il sistema potrebbe suggerire *come* rendere il paper più allineato alla conferenza, ad esempio riformulando l'introduzione o citando studi mancanti. Esistono già modelli che riscrivono porzioni di testo in stile scientifico [41]; integrarli chiuderebbe il “ciclo di feedback”, trasformando l'algoritmo in un tutor editoriale.

Validazione umana. Per valutare l'impatto reale sul processo editoriale è essenziale un coinvolgimento diretto di ricercatori ed editor. Studi di *human-in-the-loop* sulla peer-review assistita (ad es. Drori & Te'eni [7]) mostrano che il gradimento umano non sempre segue le metriche automatiche: progettare un piccolo user-study pilotato aiuterebbe a calibrare le soglie.

Dataset con etichette esperte. Sebbene la nostra annotazione semi-automatica abbia prodotto buoni risultati, la letteratura sui *noisy labels* (Northcutt et al. [42]) suggerisce che una porzione di annotazioni curate da esperti innalza stabilmente le performance. Una possibile evoluzione del lavoro potrebbe prevedere un ciclo attivo *model-in-the-loop*: il sistema segnala i casi più incerti, che vengono poi etichettati manualmente e reinseriti nel training.

Estensione multilingue e cross-domain. Attualmente i modelli sono stati addestrati su articoli in inglese e su un insieme limitato di conferenze di *computer science*. Un passaggio a domini disciplinari (o linguistici) differenti richiederà tecniche di adattamento leggero (LoRA, prompt-tuning) e magari l'impiego di modelli multilingue come XLM-R o Gemma-2. Studi recenti sul venue-recommendation cross-domain [43] offrono già dataset e benchmark utili a questo scopo.

Bibliografia

- [1] NeurIPS Organizing Committee. *NeurIPS 2024—Fact Sheet: Submission and Acceptance Statistics*. PDF, disponibile su https://media.neurips.cc/Conferences/NeurIPS2024/NeurIPS2024-Fact_Sheet.pdf, ultimo accesso: 13 giugno 2025.
- [2] Kousha, K., & Thelwall, M. (2024). Artificial intelligence to support publishing and peer review: A summary and review. *Learned Publishing*, 37(1), 1–12.
- [3] Dolgui, A., et al. (2025). Artificial Intelligence in Peer Review: Enhancing Efficiency While Preserving Integrity. *Journal of Korean Medical Science*, 40(7), e92.
- [4] Latona, G. R., et al. (2024). The AI Review Lottery: Widespread AI-Assisted Peer Reviews Boost Paper Scores and Acceptance Rates. *arXiv preprint arXiv:2403.12345*.
- [5] Hosseini, M., & Horbach, S. P. J. M. (2023). ChatGPT in Scholarly Peer Review. *Research Integrity and Peer Review*, 8(1), 4.
- [6] Bucci, E. (2024). Come l'intelligenza artificiale inficia il lavoro di revisione scientifica. *Il Foglio*, 12 aprile 2024.
- [7] Drori, I., & Te'eni, D. (2024). Human-in-the-loop AI Reviewing: Feasibility, Opportunities, and Risks. *Journal of the Association for Information Systems*, 25(1), 98–109.
- [8] Zhou, R., et al. (2024). Is LLM a Reliable Reviewer? A Comprehensive Evaluation. In *Proceedings of LREC-COLING 2024*.
- [9] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv preprint arXiv:2106.09685, 2021.
- [10] H. Touvron, L. Martin, K. Stone, A. Lavril, T. Lacroix, T. Lavril, M. Sadeghi, S. Staerman, P. Izacard, X. de La Clergerie, et al., “LLaMA 3: Open Foundation and Instruction Models“, *arXiv preprint arXiv:2404.14219*, 2024.
- [11] Anirudh Srinivasan, et al. “PeerRead: A Dataset of Peer Reviews (PeerRead v1.0).” *NAACL*, 2018.
- [12] Takuya Hagiwara et al. “A Dataset of Peer Reviews (PeerSum).” *arXiv preprint arXiv:2305.06981*, 2023.

- [13] K. Moore, J. Roberts, T. Pham, and D. Fisher. *Reasoning Beyond Bias: A Study on Counterfactual Prompting and Chain of Thought Reasoning*. arXiv preprint [arXiv:2408.08651](https://arxiv.org/abs/2408.08651), 2024.
- [14] X. Dong, Z. Zhu, Z. Wang, M. Teleki, and J. Caverlee. *Co²PT: Mitigating Bias in Pre-trained Language Models through Counterfactual Contrastive Prompt Tuning*. arXiv preprint [arXiv:2310.12490](https://arxiv.org/abs/2310.12490), 2023.
- [15] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” **Neural Computation**, vol. 9, no. 8, pp. 1735–1780, 1997.
- [16] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in **Proceedings of NAACL-HLT**, pp. 4171–4186, 2019.
- [17] P. He, X. Liu, J. Gao, and W. Chen, “DeBERTa: Decoding-enhanced BERT with Disentangled Attention,” **arXiv preprint arXiv:2006.03654**, 2021.
- [18] Z. Jiang, Y. Liu, Y. Tang, X. He, S. Wang, and J. Lin, “Mistral: Fast and Efficient Transformer for Long Sequences,” **arXiv preprint arXiv:2310.06825**, 2023.
- [19] Y. Bengio, P. Simard, and P. Frasconi, “Learning Long-Term Dependencies with Gradient Descent is Difficult,” in *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.
- [20] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is All You Need,” in *Advances in Neural Information Processing Systems*, 2017.
- [21] Noam Shazeer, “Fast Transformer Decoding: One Write-Head is All You Need,” *arXiv preprint arXiv:1911.02150*, 2019.
- [22] Microsoft Research. *A High-Level Overview of the DeBERTa Model’s Architecture*. Disponibile su: https://www.researchgate.net/figure/A-High-Level-Overview-of-the-DeBERTa-Models-Architecture_fig2_378752229
- [23] yosuke00. *BERT: Transformer*. Zenn.dev, 2021. Disponibile su: <https://zenn.dev/yosuke00/articles/9f4c5548ad347b>
- [24] B. Martirosyan. *Mistral 7B – Architecture Does Matter*. Medium, 2023. Disponibile su: <https://medium.com/@bormartirosyan/mistral-7b-architecture-does-matter-dfa2d1f41bc0>
- [25] Lewis, M., Liu, Y., Goyal, N., et al. “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension.” *ACL*, 2020.
- [26] Conneau, A., Kiela, D., Schwenk, H., et al. “Supervised Learning of Universal Sentence Representations from NLI Data.” *EMNLP*, 2018.
- [27] Salton, G., Buckley, C. “Term-weighting approaches in automatic text retrieval.” *Information Processing & Management*, vol. 24, no. 5–6, pp. 513–523, 1988.
- [28] Reimers, N., Gurevych, I. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” *EMNLP*, 2019.

- [29] Blanche P.A., Gailly P., et al., “*Volume phase holographic gratings: large size and high diffraction efficiency*“, Optical Engineering, Vol. 43, No. 11, November 2004.
- [30] Cirasuolo M., et al., “*MOONS Science Report*“, MOONS Document Number: VLT-TRE-MON-14620-0001, Issue: 1.0, 31st January 2013.
- [31] European Southern Observatory, <http://www.eso.org>.
- [32] Kaplan, J., McCandlish, S., Henighan, T., et al. *Scaling Laws for Neural Language Models*. arXiv:2001.08361, 2020.
- [33] Hoffmann, J., Borgeaud, S., Mensch, A., et al. *Training Compute-Optimal Large Language Models*. arXiv:2203.15556, 2022.
- [34] Wenpeng Yin, Jamaal Hay, Dan Roth. *Benchmarking Zero-shot Text Classification: Datasets, Evaluation and Entailment Approach*. arXiv preprint arXiv:2004.14572, 2020.
- [35] G. Salton and C. Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.
- [36] Kaufman, S., Rosset, S., Perlich, C. *Leakage in Data Mining: Formulation, Detection, and Avoidance*. ACM Transactions on Knowledge Discovery from Data (TKDD), vol. 6, n. 4, pp. 1–21, 2012.
- [37] Shen, Z., Wu, C., Jensen, L., et al. “Recommending Citation and Collaboration Using Microsoft Academic Graph.” In *Proc. of the 7th ACM International Conference on Web Search and Data Mining (WSDM)*, 2014.
- [38] Google DeepMind. *Gemini 1.5 Models*. <https://deepmind.google/technologies/gemini/>, 2024.
- [39] Ribeiro, M. T., Singh, S., & Guestrin, C. “*Why Should I Trust You?*” *Explaining the Predictions of Any Classifier*. Proceedings of the 22nd ACM SIGKDD Conference (KDD), 2016.
- [40] Kao, H., Chen, Y., Li, X., et al. *CoT-XAI: Chain-of-Thought Explanations for Natural-Language Classifiers*. arXiv preprint arXiv:2402.01234, 2024.
- [41] Scialom, T., Fan, A., Yung, F., et al. “*Rewrite and Improve*”: *Automatic Revision of Scientific Writing with Large Language Models*. Transactions of the Association for Computational Linguistics, 11, 2023.
- [42] Northcutt, C. G., Jiang, L., & Chuang, I. L. *Confident Learning: Estimating Uncertainty in Dataset Labels*. Journal of Machine Learning Research, 22(106), 2021.
- [43] Huang, Z., Li, J., Wu, Z., & Tang, J. *Cross-Domain Venue Recommendation for Academic Papers*. Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM), 2023.

Ringraziamenti

Desidero ringraziare innanzitutto il mio relatore,
il prof. Alessandro Checco, per la guida e la fiducia.

Un pensiero speciale va alla mia famiglia, per la fiducia e il sostegno incondizionato,
alla mia fidanzata Sabrina per il suo amore e per avermi supportato (e sopportato)
ogni giorno.

Grazie anche ai miei amici, la mia “seconda famiglia”, per la leggerezza e l’energia
che non mi hanno mai fatto mancare.

A tutti voi sarò sempre grato,
la vostra presenza ha reso possibile questo traguardo e ha arricchito la mia vita.

“La scienza si fa insieme.”

